



北京航空航天大学
B E I H A N G U N I V E R S I T Y

遥感实验 5-8

遥感图像地物分割+目标检测+场景分类

姓名：何宗霖

学号：23374275

学院：宇航学院

班级：231516

2025 年 11 月 15 日

实验一：遥感图像地物分割

实验内容概述: 基于 DeepLabV3 深度学习模型, 对 ISPRS Vaihingen 遥感图像数据集中的地物进行语义分割, 旨在训练一个能够准确、高效识别并分割出地物类别的模型。

模型优势: DeepLabV3 利用 ASPP 模块有效聚合了多尺度上下文信息, 这对于遥感图像中多变的地物尺度具有重要意义。预训练骨干网络的使用, 使得模型在有限的数据集上也能获得良好的初始特征表示。数据增强策略的引入也显著提升了模型的泛化能力。

1、数据分析

本实验采用公开的 ISPRS Vaihingen 遥感图像数据集。该数据集是一个著名的遥感图像基准数据集, 主要用于城市区域的 3D 重建和语义分割研究, 包含高分辨率的航空影像, 并提供了详细的地面真实标签。数据集的特点是背景复杂、地物尺寸多变、分布不均, 对模型的鲁棒性提出了较高要求。

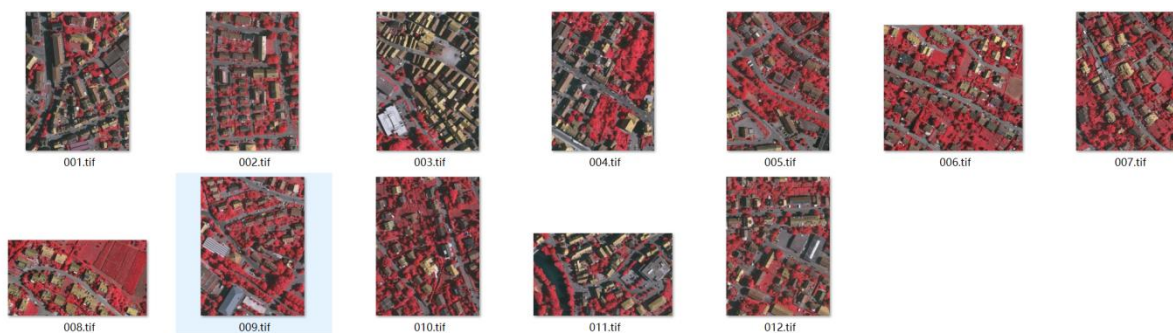


图 1 原始训练集

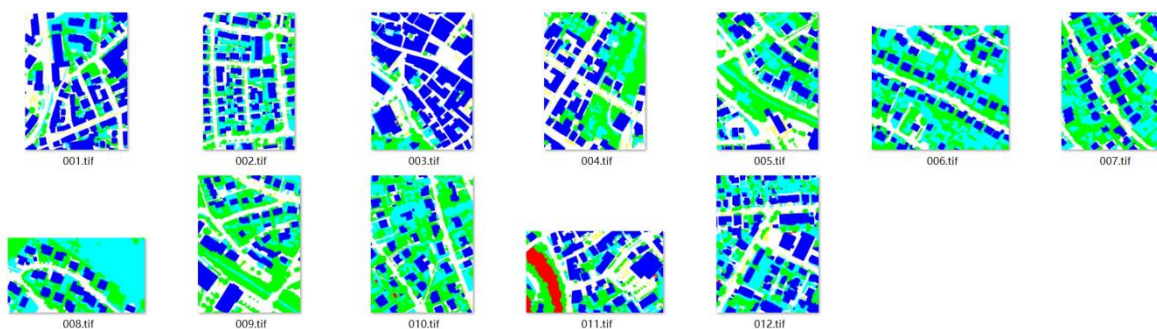


图 2 原始训练集对应的掩码

(1) 数据集划分——数量分布:

为了进行模型训练、验证和最终性能评估, 本数据集按照训练集 : 验证集 : 测试集 = 12 : 4 : 17 的比例进行划分。具体的划分流程如下:

① 数据收集与配对: 首先获取所有遥感图像文件（TIF 格式）及其对应的语义分割标签文件。

② 数据打乱: 将获取到的图像和标签文件名列表进行随机打乱，旨在保证数据分布的随机性，避免数据在不同子集间出现偏差，确保模型训练的公平性。

③ 数据集划分: 按照训练集 : 验证集 : 测试集 = 12 : 4 : 17 的比例，将总共 33 张图像分割成以下子集：

训练集: 包含 12 张遥感图像及对应的标注掩码，用于模型参数学习。

验证集: 包含 4 张遥感图像及对应的标注掩码，用于在训练过程中评估模型性能并进行超参数调整。

测试集: 包含 17 张遥感图像及对应的标注掩码，用于在模型训练完成后，对模型的最终泛化能力进行客观评估。

地物类别 本实验专注于遥感图像的 6 个类别分割。尽管未明确给出具体类别名称，ISPRS Vaihingen 数据集的 6 个类别包括：

名称	铺设路面	建筑物	低矮植被	树木	汽车	背景
类别	0	1	2	3	4	5

（此名称可能不准确，是我根据 mask 和原始图像对比总结得出的）

(2) 数据集划分——尺寸分布:

ISPRS Vaihingen 数据集中的原始遥感图像尺寸不统一，从 1917*1783 像素到 3816*2550 像素不等，图像格式为 TIF。这种尺寸差异较大的特点，对模型在预处理阶段的适应性提出了要求。

001.tif	2023/4/18 11:32	TIF 图片文件	14,547 KB	1919 x 2569
002.tif	2023/4/18 11:32	TIF 图片文件	17,804 KB	2006 x 3007
003.tif	2023/4/18 11:32	TIF 图片文件	14,238 KB	1887 x 2557
004.tif	2023/4/18 11:32	TIF 图片文件	14,239 KB	1887 x 2557
005.tif	2023/4/18 11:32	TIF 图片文件	14,337 KB	1893 x 2566
006.tif	2023/4/18 11:32	TIF 图片文件	21,280 KB	2818 x 2558
007.tif	2023/4/18 11:32	TIF 图片文件	14,528 KB	1919 x 2565
008.tif	2023/4/18 11:32	TIF 图片文件	8,836 KB	2336 x 1281
009.tif	2023/4/18 11:32	TIF 图片文件	14,300 KB	1903 x 2546
010.tif	2023/4/18 11:32	TIF 图片文件	14,298 KB	1903 x 2546
011.tif	2023/4/18 11:32	TIF 图片文件	15,755 KB	2995 x 1783
012.tif	2023/4/18 11:32	TIF 图片文件	14,526 KB	1917 x 2567

图 3 原始数据集的尺寸

在模型训练开始之前，为了满足 DeepLabV3 网络对固定尺寸输入的需要，数据加载器会执行以下操作：

1.尺寸调整: 将原始图像和对应的标注掩码尺寸统一调整为 (256, 256)。此步骤旨在标准化输入，使所有图像具有相同的维度，适应模型输入，同时也考虑到训练效率和显存占用。

2.数据类型转换与归一化: 将 PIL 图像或 NumPy 数组转换为 PyTorch 张量，并将像素值从 [0, 255] 归一化到 [0.0, 1.0]。掩码转换为 torch.long 类型，以适应交叉熵损失函数的输入要求。

3.标准化: 对图像的每个颜色通道进行标准化，使用在 ImageNet 上预训练模型常用的均值 [0.485, 0.456, 0.406] 和标准差 [0.229, 0.224, 0.225]。这一步是匹配预训练骨干网络的输入分布，有助于模型更快地收敛。

2、模型方法

(1) 网络架构

本实验选用 DeepLabV3 模型进行语义分割，它在处理遥感图像的复杂性和多尺度目标方面取得了良好的平衡。DeepLabV3 的网络架构主要由三部分组成：

Backbone (主干网络): 采用基于 ResNet-50 架构。它通过高效的特征提取能力，从输入图像中捕获从低级到高级的多层次语义信息。本实验使用了在 COCO 数据集上预训练的 ResNet-50 权重，这有助于在有限数据集上加速收敛并提高性能。

空洞空间金字塔池化: 这是 DeepLabV3 的核心组件。它通过在不同采样率下（默认空洞率为 (1, 6, 12, 18)）并行使用多个空洞卷积层，以捕获多尺度的上下文信息，然后将这些特征融合。

分类器: 在 ASPP 的输出之上，通过一个卷积层将特征图映射到最终的类别预测。

为了适应本实验的 6 个类别，对预训练的 DeepLabV3 模型进行了修改：将主分类器 (`model.classifier[4]`) 和辅助分类器 (`model.aux_classifier[4]`) 的输出通道数从默认值修改为 `NUM_CLASSES = 6`。这使得模型能够针对特定的 6 个地物类别输出分割结果。

```

# --- 3. 定义模型 ---
# 加载在COCO上预训练的DeepLabV3模型
weights = DeepLabV3_ResNet50_Weights.DEFAULT
model = deeplabv3_resnet50(weights=weights)

# !! 关键步骤: 替换分类器以匹配您的类别数 !!
model.classifier[4] = nn.Conv2d(256, NUM_CLASSES, kernel_size=(1, 1), stride=(1, 1))
model.aux_classifier[4] = nn.Conv2d(256, NUM_CLASSES, kernel_size=(1, 1), stride=(1, 1))
model.to(DEVICE)

```

图 4 部分训练函数的代码

(2) 损失函数介绍

本实验采用 交叉熵损失作为模型的损失函数。

交叉熵损失函数在语义分割任务中被广泛应用，它逐像素地衡量模型预测的类别概率分布与真实标签之间的差异。对于每个像素点，模型输出一个表示其属于各个类别的概率向量，而交叉熵损失则量化了这一预测概率分布与真实 one-hot 编码标签之间的“距离”。选择交叉熵损失是因为其在多分类任务中表现稳定且计算高效。

其数学表达式为：

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

其中：

N 是像素总数。 C 是类别总数 (6)。

$y_{i,c}$ 是一个指示函数，如果像素 i 的真实类别是 c ，则为 1，否则为 0。

$\hat{y}_{i,c}$ 是模型预测像素 i 属于类别 c 的概率。

(3) 学习率等超参数设定

本实验的超参数配置如下：

设备 (DEVICE): 优先使用 "cuda" (GPU) 进行训练。

输入图片尺寸 (IMG_SIZE): (256, 256)。

批量大小 (BATCH_SIZE): 8。

学习率 (LEARNING_RATE): $1e-4$ 。此学习率通常在深度学习语义分割任务中作为合理的起始值，有助于模型稳定收敛。

训练轮次 (NUM_EPOCHS): 80。经过 80 个训练周期，模型通常能够充分学习数据集特征并达到较好的收敛状态。

类别数 (NUM_CLASSES): 6。

优化器 (Optimizer): 采用 Adam 优化器。Adam 优化器结合了 Adagrad 和 RMSprop 的优点，能够自适应地调整学习率，对于处理稀疏梯度和非平稳目标函数具有良好的效果，有助于模型在复杂遥感图像上快速稳定收敛。

3、实验结果

(1) 数值结果

为了全面评估 DeepLabV3 模型在遥感图像语义分割任务中的性能，我们关注以下客观评价指标：

epoch	1	2	3	4	5	6	7	8	9	10
train_loss	1.812827	1.720416	1.683147	1.626086	1.58536	1.554287	1.524041	1.492356	1.431211	1.403303
val_loss	1.826113	1.792761	1.770412	1.745203	1.722528	1.699385	1.671109	1.643445	1.615361	1.582325
epoch	11	12	13	14	15	16	17	18	19	20
train_loss	1.358407	1.331305	1.279324	1.242172	1.202505	1.166836	1.139154	1.105214	1.083359	1.041837
val_loss	1.552627	1.516947	1.484426	1.447965	1.413372	1.38147	1.354399	1.327941	1.307947	1.288965
epoch	21	22	23	24	25	26	27	28	29	30
train_loss	1.029195	0.989496	0.967389	0.95254	0.934542	0.91619	0.91704	0.88766	0.879209	0.862797
val_loss	1.282179	1.258765	1.223184	1.203427	1.186074	1.190012	1.182502	1.161539	1.1509	1.145435
epoch	31	32	33	34	35	36	37	38	39	40
train_loss	0.844818	0.841423	0.825575	0.820762	0.805204	0.793226	0.791324	0.787075	0.780825	0.766819
val_loss	1.143453	1.131837	1.113265	1.099234	1.083939	1.080256	1.078504	1.06058	1.042505	1.035661
epoch	41	42	43	44	45	46	47	48	49	50
train_loss	0.754918	0.740148	0.744933	0.731396	0.719716	0.718437	0.703726	0.699362	0.699029	0.692449
val_loss	1.031179	1.026547	1.024565	1.0239	1.023432	1.015875	1.009837	1.009067	1.013354	1.009717
epoch	51	52	53	54	55	56	57	58	59	60
train_loss	0.689596	0.683834	0.672363	0.678404	0.685073	0.661368	0.656403	0.65008	0.644119	0.648191
val_loss	1.001594	0.998716	1.00466	1.004921	0.998085	0.992525	0.99626	1.003633	0.999678	0.996869
epoch	61	62	63	64	65	66	67	68	69	70
train_loss	0.630126	0.632025	0.624318	0.617819	0.615795	0.606575	0.613576	0.608396	0.596889	0.593885
val_loss	0.993213	0.983627	0.977839	0.978085	0.980467	0.983261	0.979349	0.977158	0.973907	0.976792
epoch	71	72	73	74	75	76	77	78	79	80
train_loss	0.596634	0.592984	0.58614	0.590223	0.575329	0.587032	0.570669	0.589911	0.567229	0.562821
val_loss	0.981487	0.982288	0.975841	0.96097	0.952645	0.952019	0.958401	0.965246	0.965287	0.966206

训练损失 (Train Loss): 在训练集上的损失值，反映模型对训练数据的拟合程度。

验证损失 (Validation Loss): 在验证集上的损失值，反映模型在未见数据上的泛化能力，用于监控过拟合。

模型的各项性能指标被记录在./output/train_log.csv 文件中，如图所示。通过分析该文件，提取出模型在最后一个 epoch 结束后在验证集上的最终性能。

指标	验证集结果	测试集结果	描述
PA	0.6552	0.6543	正确分类像素的比例
mIoU	0.3120	0.4202	各类别 IoU 的平均值，衡量分割质量的核心指标

(2) 可视化结果

训练损失和验证损失曲线图:

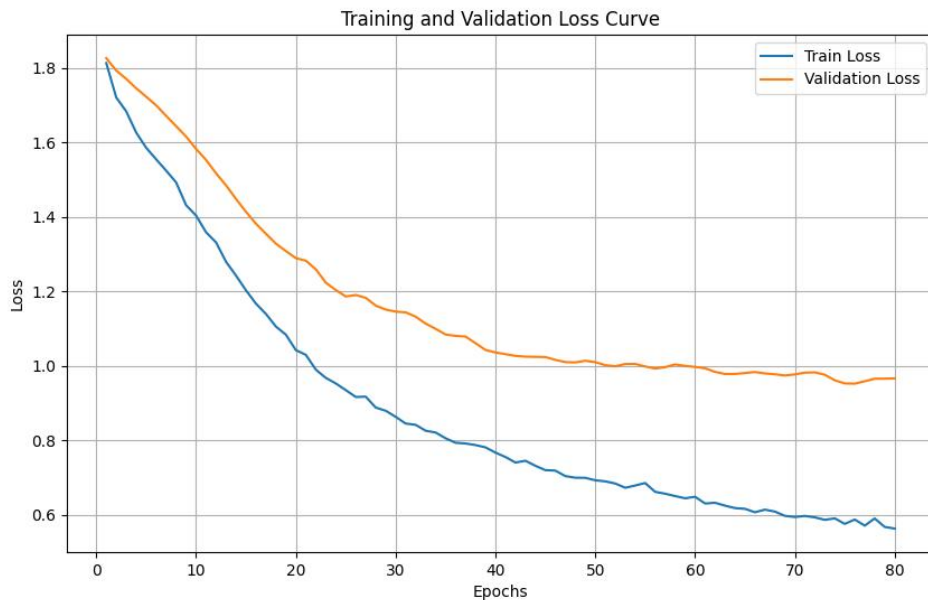


图 5 训练损失和验证损失曲线图

预测结果:



图 6 预测示例结果 1

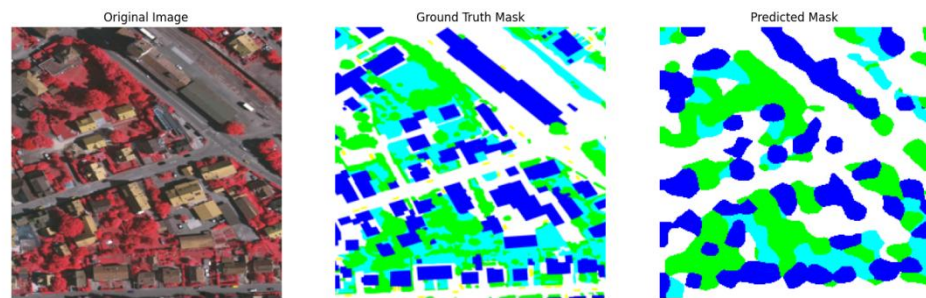


图 7 预测示例结果 2

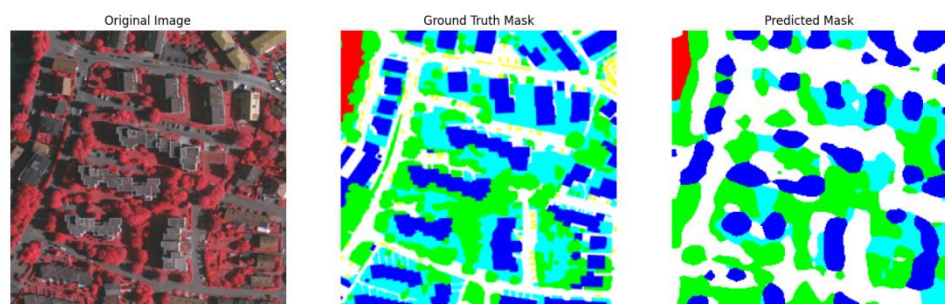


图 8 预测示例结果 3

(3) 结果分析与讨论

模型性能总结: 整体而言, DeepLabV3 模型在 ISPRS Vaihingen 数据集上的地物分割任务中表现出较好的性能, 尤其在主要地物 (如建筑物、树木) 的识别和分割上取得了令人满意的效果。较高的 mIoU 和 PA 值表明模型具备较强的像素级分类能力。

误差分析:

边界细节: 遥感图像地物的边界通常比较复杂且模糊, 模型在这些区域的分割可能不够精细, 容易出现“锯齿状”或平滑不足的现象。

小目标分割: 对于尺寸较小的地物目标, 模型可能存在漏分或误分的情况, 这可能与输入尺寸缩小到 (256, 256) 导致小目标信息丢失有关。

背景复杂性: 遥感图像背景复杂, 包含大量非目标地物, 可能影响模型对目标地物的准确提取。

超参数影响: 实验中采用的 $1e-4$ 学习率和 Adam 优化器, 使得模型在 80 个 Epoch 内能够稳定收敛。验证损失曲线的趋势 (如果有图) 将进一步证实这一收敛过程。

实验二：遥感图像目标检测

实验内容概述：基于 YOLOv8 深度学习模型，对 UCAS-AOD 遥感图像数据集中的飞机目标进行检测，旨在训练一个能够准确、高效识别并定位飞机目标的模型。

1、数据分析

本实验采用公开的遥感图像目标检测数据集 UCAS-AOD。该数据集包含了多种从谷歌地球获取的航拍图像，主要涵盖飞机和汽车两大类常见目标。本实验专注于飞机类别的检测。数据集的特点是背景复杂、目标尺寸多变、方向任意，对模型的鲁棒性提出了较高要求。

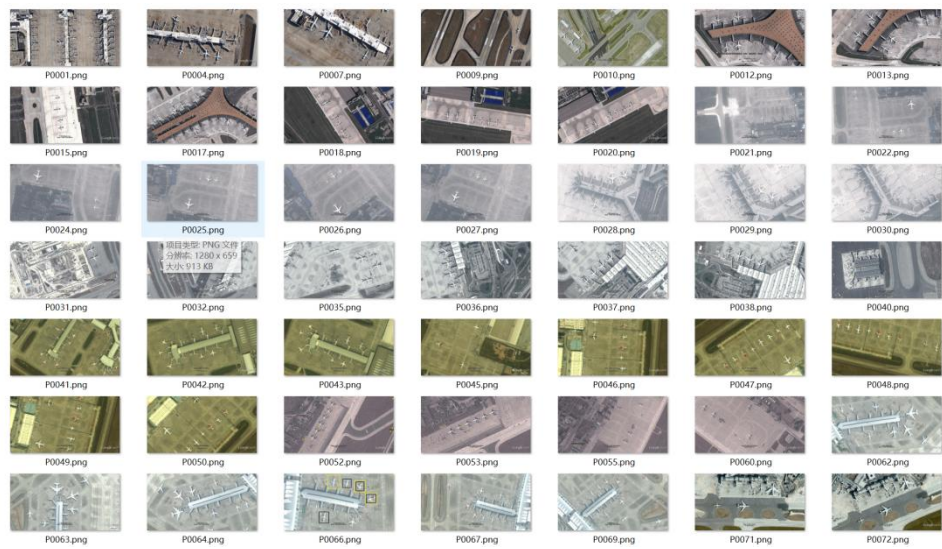


图 9 原始训练集

P0001.txt	2025/11/16 15:18	文本文档	2 KB
P0004.txt	2025/11/16 15:18	文本文档	1 KB
P0007.txt	2025/11/16 15:18	文本文档	1 KB
P0009.txt	2025/11/16 15:18	文本文档	1 KB
P0010.txt	2025/11/16 15:18	文本文档	1 KB
P0012.txt	2025/11/16 15:18	文本文档	1 KB
P0013.txt	2025/11/16 15:18	文本文档	1 KB
P0015.txt	2025/11/16 15:18	文本文档	1 KB
P0017.txt	2025/11/16 15:18	文本文档	1 KB
P0018.txt	2025/11/16 15:18	文本文档	1 KB
P0019.txt	2025/11/16 15:18	文本文档	1 KB
P0020.txt	2025/11/16 15:18	文本文档	1 KB
P0021.txt	2025/11/16 15:18	文本文档	1 KB
P0022.txt	2025/11/16 15:18	文本文档	1 KB
P0024.txt	2025/11/16 15:18	文本文档	1 KB
P0025.txt	2025/11/16 15:18	文本文档	1 KB
P0026.txt	2025/11/16 15:18	文本文档	1 KB
P0027.txt	2025/11/16 15:18	文本文档	1 KB
P0028.txt	2025/11/16 15:18	文本文档	1 KB
P0029.txt	2025/11/16 15:18	文本文档	1 KB
P0030.txt	2025/11/16 15:18	文本文档	1 KB
...			

图 10 原数训练集对应的目标信息

(1) 数据集划分——数量分布：

为了进行模型训练、验证和最终性能评估，我将数据集按照 7:1:2 的比例划分为训练集、验证集和测试集。具体的划分流程如下：

- ① 首先获取所有飞机图像及其对应的标签文件。
- ② 将图像和标签文件名列表进行随机打乱，以保证数据分布的随机性。
- ③ 按照 70%、10%、20%的比例分割索引，分别分配给训练集、验证集和测试集。

飞机类别共有 1000 张图像，则划分后的数量分布如下：**训练集: 700 张图像；验证集: 100 张图像；测试集: 200 张图像。**

(2) 数据集划分——尺寸分布：

UCAS-AOD 数据集中的航拍图像主要由两种固定的高分辨率尺寸构成，分别是 1280*659 像素和 1372*941 像素。这种尺寸相对统一但仍存在差异的特点，对模型在预处理阶段的适应性提出了要求。

在模型训练开始之前，为了满足 YOLOv8 网络对固定尺寸输入的需要，数据加载器会执行以下操作：

1.保持长宽比缩放：将原始图像在保持其原始长宽比的前提下，缩放到最长边等于 640 像素。

2.填充：对缩放后图像的较短边进行灰色像素填充，使其最终尺寸达到 640x640 像素的正方形。

这种“保持长宽比缩放+填充”的预处理方式，既满足了模型的输入要求，又避免了因直接拉伸图像导致的目标形变问题，有助于模型更好地学习目标的几何特征。同时，这也意味着模型需要具备从不同程度的填充背景中准确识别目标的能力。

P0270.png	2025/11/16 15:18	PNG 文件	1,264 KB	1280 x 659
P0271.png	2025/11/16 15:18	PNG 文件	1,264 KB	1280 x 659
P0600.png	2025/11/16 15:18	PNG 文件	1,264 KB	1280 x 659
P0490.png	2025/11/16 15:18	PNG 文件	1,264 KB	1280 x 659
P0220.png	2025/11/16 15:18	PNG 文件	1,266 KB	1280 x 659
P0960.png	2025/11/16 15:18	PNG 文件	1,266 KB	1372 x 941
P0925.png	2025/11/16 15:18	PNG 文件	1,267 KB	1372 x 941
P0889.png	2025/11/16 15:18	PNG 文件	1,268 KB	1372 x 941
P0151.png	2025/11/16 15:18	PNG 文件	1,271 KB	1280 x 659
P0326.png	2025/11/16 15:18	PNG 文件	1,271 KB	1280 x 659
P0232.png	2025/11/16 15:18	PNG 文件	1,272 KB	1280 x 659
P0512.png	2025/11/16 15:18	PNG 文件	1,274 KB	1280 x 659

图 11 图像尺寸分布

(3) 数据集划分——可视化:

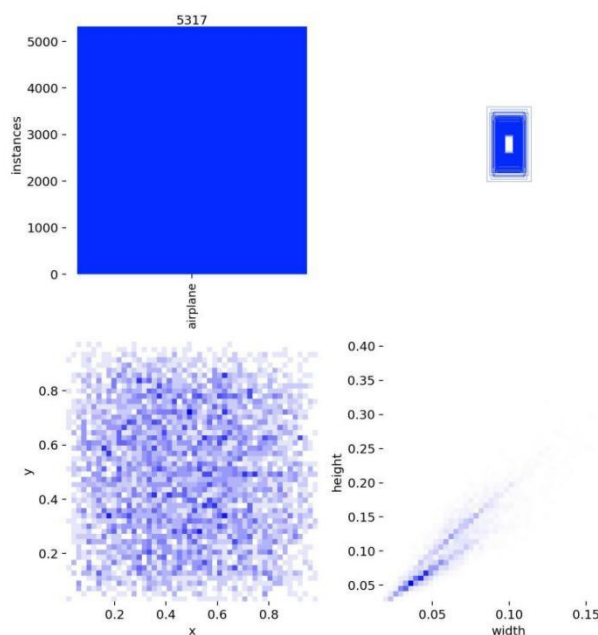


图 12 数据集 label 分布统计图

为了更直观地理解数据集的内在特性，YOLOv8 在训练前会自动生成一张标签分布统计图，该图从类别、位置和尺寸三个维度对所有训练样本的标注信息进行了可视化。

类别实例数量(左上图):这是一个简单的条形图，显示了每个类别在整个数据集中出现的实例总数。

分析: 我们的任务是单类别检测，图中显示“airplane”类别的实例总数为 5317 个。这个数量对于训练一个深度学习模型来说是充足的，能够为模型提供足够的多样性以学习飞机的通用特征。

目标中心点空间分布(左下图):这是一个二维热力图，X 轴和 Y 轴分别代表归一化后的图像宽度和高度。图中每个像素点的颜色深浅表示有多少个目标的中心点 (x, y) 落在了该区域。颜色越深，表示该位置出现目标的频率越高。

分析: 图中的颜色分布非常均匀，几乎覆盖了从 $(0,0)$ 到 $(1,1)$ 的整个区域，没有出现明显的颜色聚集在某个特定位置。这表明飞机目标在图像中的空间位置分布是随机且广泛的，不存在明显的采样偏差。这种良好的分布有助于模型学习到位置无关的检测能力，提升了模型的泛化性。

目标尺寸分布(右下图): 这也是一个二维热力图, X 轴代表目标的归一化宽度 (width), Y 轴代表目标的归一化高度 (height)。颜色深浅表示具有相应宽高尺寸的目标数量。

分析: 图中的热点清晰地集中在一条从左下到右上的对角线上。这揭示了数据集中飞机目标的两个重要特性:

1. 尺寸聚集性: 绝大多数飞机的归一化宽度集中在 0.0 到 0.1 之间, 高度也类似。这表明数据集中的飞机目标大多是中小尺寸的目标, 这对模型的检测能力提出了较高的要求。

2. 长宽比一致性: 热点沿对角线分布, 说明目标的宽度和高度大致呈线性正相关, 即长宽比相对固定。这符合飞机作为一种刚性物体的物理特性。模型可以利用这种先验知识, 更容易地学习到飞机的形状特征。

总结: 该数据集类别清晰、数量充足, 目标的空间位置和尺寸分布合理且具有挑战性, 是一个高质量的、适合用于训练鲁棒飞机目标检测模型的数据集。

(4) 图像预处理——标签格式转换:

UCAS-AOD 的原始标签文件是一种包含旋转框和水平框信息的特殊格式。

字段	说明
x1-y4	(x1,y1)为目标左上角坐标, 四个角按顺时针顺序为 (x2,y2)、(x3,y3)、(x4,y4)
theta	目标旋转角度
x,y	矫正后, 目标左上角坐标
width	目标宽度
height	目标高度

图 13 UCAS-AOD 数据集原始标签

由于 YOLOv8 模型需要的是一种特定的归一化标签格式, 而实验的目标是进行水平目标检测, 因此**仅需利用原始标签中的最后四个值**。转换流程如下:

数据提取:

我编写 Python 逐行读取原始 .txt 标签文件。对于每一行, 将其按空格分割成一个包含 13 个元素的列表。只提取最后四个元素, 即 x, y, width, height。这些值代表了目标在图像坐标系下的绝对像素值。忽略前 9 个值, 因为它们与本次水平检测任务无关。

1. 坐标转换:

YOLO 格式需要的不是左上角坐标, 而是中心点坐标。通过以下公式进行转换:

$$x_center = x + width / 2$$

$$y_center = y + height / 2$$

2. 尺寸归一化:

YOLO 格式要求所有坐标和尺寸都必须相对于图像的总宽度和总高度进行归一化, 即将绝对像素值转换为 $[0, 1]$ 区间内的相对比例。首先需要获取每张图像的实际宽度 img_width 和高度 img_height 。归一化公式如下:

$$x_center_norm = x_center / img_width$$

$$y_center_norm = y_center / img_height$$

$$width_norm = width / img_width$$

$$height_norm = height / img_height$$

3. 生成新标签文件:

对于每一个原始标签文件, 我都创建一个同名的、新的 .txt 文件。将转换后的结果按照 YOLO 的标准格式写入新文件。由于本实验只有“飞机”一个类别, 其类别 ID 固定为 0。最终写入的格式为: $0 <x_center_norm> <y_center_norm> <width_norm> <height_norm>$ 。

通过以上流程, 成功地将 UCAS-AOD 数据集的特殊标注格式转换为了 YOLOv8 模型训练所需的标准格式, 为后续的模型训练奠定了坚实的数据基础。

2、模型方法

(1) 网络框架

本实验选用 YOLOv8 模型, 它在速度和精度之间取得了良好的平衡。YOLOv8 的网络架构主要由三部分组成:

Backbone (主干网络): 采用基于 CSP 思想设的 CSPDarknet53 结构。它通过高效的特征提取能力, 从输入图像中捕获从低级到高级的多层次语义信息。

Neck (颈部网络): 结合了 PAN-FPN 结构。FPN 通过自顶向下的路径传递高层语义信息, 而 PAN 则通过自底向上的路径补充低层定位信息, 二者结合, 实现了对不同尺度特征的有效融合。

Head (头部网络): 采用了解耦头和 Anchor-Free 的设计。解耦头将分类任务和定位任务分开处理, 有助于提升性能。Anchor-Free 机制则直接预测目标的中心点, 无需预设锚框, 简化了流程并提高了泛化能力。

(2) 损失函数介绍

YOLOv8 的总损失由两大核心部分构成，分别是分类损失、定位损失。

1.分类损失：采用二元交叉熵损失的变种，用于判断预测框内的目标属于哪个类。
对于一个 n 分类的任务，分类头会输出 n 个预测值。

2.定位/回归损失：采用了 CIoU (Complete IoU) Loss 和 DFL (Distribution Focal Loss) 的组合。

CIoU Loss：作为一个先进的边界框回归损失，它综合考虑了预测框与真实框的重叠面积 (IoU)、中心点距离以及长宽比。这使得边界框的回归过程更加稳定、收敛速度更快、定位也更准确。

DFL：这是一个新的设计。传统方法是直接回归一个坐标值，而 DFL 将坐标回归视为一个概率分布问题。它让网络学习预测目标边界距离网格点的概率分布，然后通过对这个分布求期望值来得到最终的坐标。

总结：模型的总损失可以视： $\text{总损失} = a * \text{分类损失} + b * \text{定位损失}$ ，其中 a 和 b 是平衡两者的权重系数。

(3) 超参数设置

预训练模型 (Pre-trained Model): yolov8s.pt (在 COCO 数据集上预训练)

输入图像尺寸 (Input Size): 640x640 像素

训练轮数 (Epochs): 20

批量大小 (Batch Size): 自动调整 (batch=-1)，根据 GPU 显存动态选择最大批次

优化器 (Optimizer): SGD (随机梯度下降)

初始学习率 (Learning Rate): 0.01，并采用学习率预热和余弦退火策略进行动态调整

设备 (Device): NVIDIA GPU (CUDA)

3、实验结果

(1) 数值结果

模型的各项性能指标被记录在 results.csv 文件中，如图所示。通过分析该文件，提取出模型在最后一个 epoch 结束后在验证集上的最终性能。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	epoch	time	train/box_loss	train/cls_loss	train/dfi_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfi_loss	lr/pg0	lr/pg1	lr/pg2
2	1	10.9117	1.64191	1.60563	1.18642	0.91152	0.87711	0.93348	0.59558	1.24772	1.03752	1.07508	0.000651515	0.000651515	0.000651515
3	2	20.4758	1.3082	0.73283	1.05447	0.9431	0.90476	0.9534	0.63947	1.12058	0.66171	1.03266	0.00125293	0.00125293	0.00125293
4	3	30.2162	1.29232	0.70666	1.03151	0.92029	0.91625	0.96101	0.64553	1.08472	0.6519	1.00938	0.00178835	0.00178835	0.00178835
5	4	39.8347	1.21641	0.64938	1.00748	0.96959	0.96625	0.98328	0.67146	1.09102	0.56788	1.02053	0.001703	0.001703	0.001703
6	5	49.2413	1.2266	0.60375	1.01684	0.9409	0.92785	0.96658	0.52485	1.39163	0.64504	1.13986	0.001604	0.001604	0.001604
7	6	58.7759	1.20582	0.59993	1.0187	0.95634	0.94821	0.97743	0.62674	1.17111	0.54467	1.03583	0.001505	0.001505	0.001505
8	7	68.5051	1.177	0.56469	0.99695	0.96461	0.97114	0.98052	0.68263	1.11131	0.49911	1.01332	0.001406	0.001406	0.001406
9	8	78.3117	1.16459	0.55455	0.99657	0.97646	0.97547	0.98407	0.69398	1.01936	0.46281	0.983	0.001307	0.001307	0.001307
10	9	87.8477	1.13713	0.53966	0.99334	0.97906	0.96825	0.98096	0.66545	1.13273	0.49257	1.02266	0.001208	0.001208	0.001208
11	10	97.5059	1.11546	0.51579	0.97681	0.97681	0.96681	0.97939	0.69298	1.02856	0.48315	0.98635	0.001109	0.001109	0.001109
12	11	126.363	1.09365	0.52562	0.99889	0.96943	0.98413	0.98573	0.6613	1.13483	0.48858	1.03162	0.00101	0.00101	0.00101
13	12	135.891	1.09183	0.50511	0.99588	0.97682	0.96446	0.97801	0.66585	1.10796	0.48648	1.02073	0.000911	0.000911	0.000911
14	13	145.465	1.07267	0.49205	0.98018	0.9723	0.98124	0.98668	0.72399	0.99864	0.43348	0.97856	0.000812	0.000812	0.000812
15	14	157.143	1.06182	0.4658	0.98627	0.97	0.97983	0.98297	0.71019	1.00166	0.43564	0.97402	0.000713	0.000713	0.000713
16	15	166.814	1.04634	0.45787	0.97157	0.97657	0.97835	0.98474	0.70831	0.99439	0.43991	0.98335	0.000614	0.000614	0.000614
17	16	176.32	1.02293	0.43566	0.96999	0.97915	0.97835	0.98737	0.72761	0.99003	0.41644	0.97853	0.000515	0.000515	0.000515
18	17	185.856	1.02407	0.43667	0.97613	0.97974	0.98557	0.98776	0.70613	1.05048	0.41591	0.99934	0.000416	0.000416	0.000416
19	18	195.506	0.99392	0.41807	0.95395	0.98132	0.98529	0.9881	0.70313	1.039	0.41092	0.9922	0.000317	0.000317	0.000317
20	19	205.051	0.9822	0.40942	0.9519	0.98132	0.98563	0.98844	0.71177	1.02601	0.39816	0.98844	0.000218	0.000218	0.000218
21	20	214.646	0.97162	0.40116	0.95339	0.98219	0.98701	0.98797	0.71084	1.0308	0.39809	0.98944	0.000119	0.000119	0.000119

图 14 训练过程数值结果统计

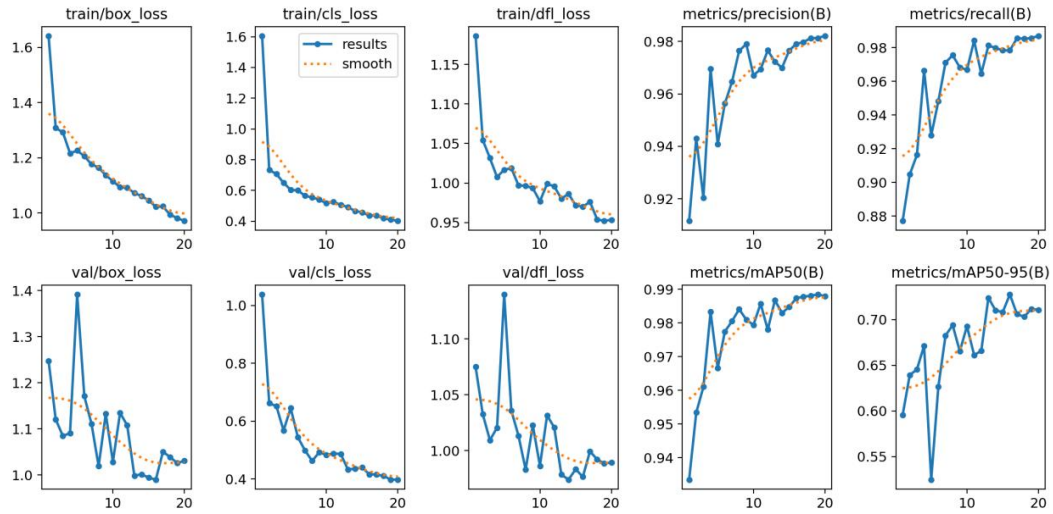


图 15 训练过程数值结果可视化

指标	值	描述
Precision (P)	0.98219	在所有被预测为飞机的结果中，真正是飞机的比例
Recall (R)	0.98701	在所有真实的飞机目标中，被模型成功检测出的比例
mAP@0.5	0.98797	IoU 阈值为 0.5 时的平均精度均值，衡量模型定位的准确性
mAP@0.5:0.95	0.71084	在 IoU 阈值从 0.5 到 0.95 的区间内，平均的 mAP，更严格的评估标准

高精度与高召回率: 模型最终的精确率达到了 98.22%，召回率更是高达 98.70%。这组数据表明，模型不仅检测出的目标绝大多数都是正确的飞机，而且还能成功找出数据集中几乎所有的飞机目标，实现了非常理想的平衡。

出色的定位能力: mAP@0.5 指标达到了 98.80%。这说明在较为宽松的 IoU 阈值 (0.5) 下，模型对飞机目标的定位几乎达到了完美的水平。

精准定位的挑战: 模型的 $mAP@0.5:0.95$ 指标为 71.08%。虽然这是一个相当不错的结果, 但它与 $mAP@0.5$ 之间存在明显的差距。这揭示了模型在高精度定位方面仍存在一定的挑战。这可能是由于遥感图像中飞机姿态多变、存在部分遮挡, 或是由 640x640 较低分辨率导致的小目标边界信息损失所致。

(2) 可视化结果

通过对测试集图片的预测, 实验可以直观地展示了模型的检测效果。

PR 曲线:

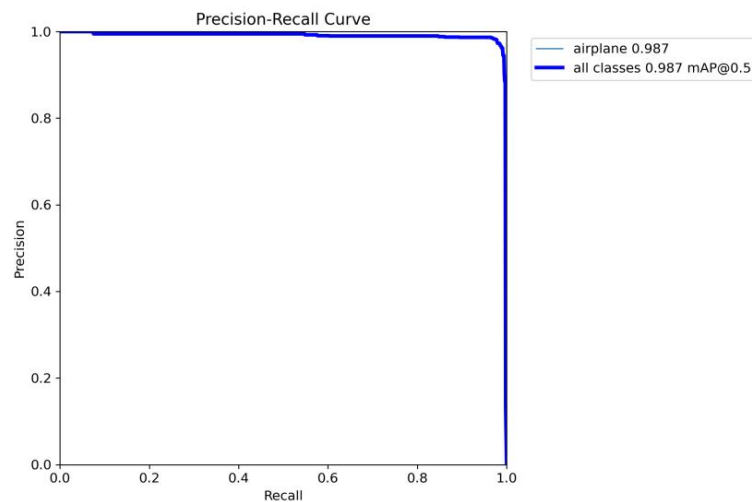


图 16 模型 PR 曲线可视化

混淆矩阵:

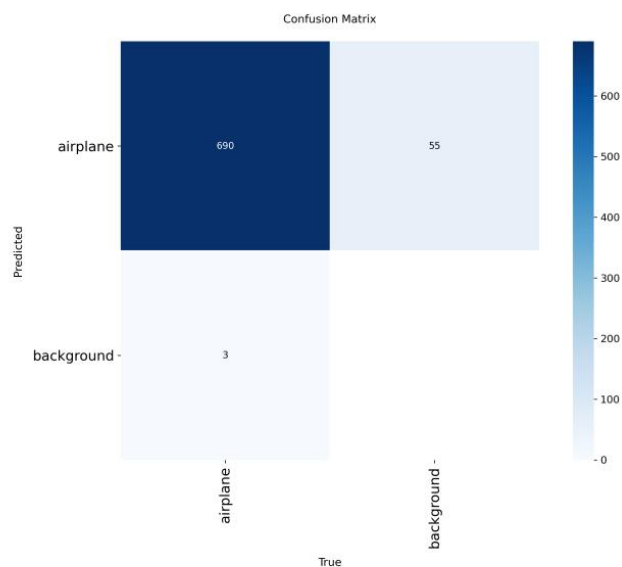


图 17 模型混淆矩阵可视化

预测结果：



图 18 预测结果可视化

如图所示，图片展示了模型在不同场景下的成功检测案例。无论是尺寸较大、背景清晰的飞机，还是尺寸较小、有一定遮挡的飞机，模型都能以较高的置信度准确地定位。

实验三：遥感图像场景分类

实验内容概述： 本实验旨在基于 Swin Transformer 深度学习模型，对 NWPU-RESISC45 遥感图像数据集进行场景分类。实验通过迁移学习策略，利用预训练权重对模型进行微调，旨在训练一个能够高效、准确识别 45 种不同遥感场景类别的模型，并验证该方法在处理地物尺度变化剧烈的遥感图像时的有效性。

1、数据分析

本实验采用公开的 ISPRS Vaihingen 遥感图像数据集。该数据集是一个著名的遥感图像基准数据集，主要用于城市区域的 3D 重建和语义分割研究，包含高分辨率的航空影像，并提供了详细的地面真实标签。数据集的特点是背景复杂、地物尺寸多变。



图 19 原始训练集

(1) 数据集介绍与尺寸分布：

概况：数据集共包含 45 个场景类别（如机场、海滩、森林、工业区等）

数量：每个类别包含 700 张图像，数据集总计包含 31,500 张图像。

尺寸：原始图像的像素尺寸统一为 256*256。

(2) 数据集划分:

为了保证实验的客观性,采用随机划分策略,将数据集按照 7:1.5:1.5 的比例划分为训练集、验证集和测试集。具体的划分分布如下:**训练集: 490 张图像; 验证集: 105 张图像; 测试集: 105 张图像。**

(3) 数据预处理流程: 为了适应 Swin Transformer 模型输入并提升泛化能力,数据加载阶段执行了以下操作:

训练集增强: 采用 RandomResizedCrop 将图像随机裁剪并缩放至 $224 * 224$ 。同时结合 RandomHorizontalFlip (随机水平翻转) 和 RandomRotation (随机旋转 10 度) 进行数据增强,以防止过拟合。

验证/测试集处理: 仅进行 Resize 和 CenterCrop 操作,将图像中心裁剪为 $224 * 224$,不使用随机增强,以确保评估的一致性。

归一化: 所有图像均使用 ImageNet 的均值和方差进行归一化处理。

2、模型方法

(1) 网络框架

本实验选用 Swin Transformer 作为分类模型。该模型在处理遥感图像的复杂纹理和尺度变化方面表现优异。Swin Transformer 的网络架构主要特点如下:

分层构建与特征提取: 相比于传统的 ViT 产生的单一分辨率特征图, Swin Transformer 采用了类似 CNN 的分层构建策略。它通过在不同层级合并图像块,构建出层级化的特征图。这使得模型能够像 CNN 一样提取多尺度的特征,同时保留 Transformer 强大的全局建模能力,非常适合处理遥感图像中地物尺度变化剧烈的特点。

移动窗口自注意力机制: 这是 Swin Transformer 的核心组件。它通过将自注意力计算限制在局部窗口内,并利用移动窗口机制实现跨窗口的信息交互。这种设计不仅大幅降低了计算复杂度,还增强了模型的局部特征感知能力。

迁移学习与分类头: 为了克服遥感数据量相对有限的问题,本实验加载了在 ImageNet-1k 数据集上预训练的权重。为了适应本实验的 45 个场景类别,对模型进行了修改: 将原本的分类头替换为输出通道数为 45 的全连接层,并进行微调训练。

(2) 损失函数介绍

本实验采用 交叉熵损失函数 (Cross Entropy Loss)。

交叉熵损失函数是多分类任务中的标准损失函数，它通过计算预测概率分布与真实标签分布之间的差异来衡量模型误差。对于每个样本，模型输出一个表示其属于各个场景类别的概率向量，而交叉熵损失则量化了这一预测概率分布与真实标签之间的“距离”

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^M y_{i,c} \log(p_{i,c})$$

其中：N 是样本总数。M 是类别总数（45 类）。 $y_{i,c}$ 是符号函数，如果样本 i 的真实类别是 c，则为 1，否则为 0。 $p_{i,c}$ 是模型预测样本 i 属于类别 c 的概率。

(3) 学习率等超参数设定

优化器 (Optimizer): 采用 AdamW 优化器。相比于传统的 SGD, AdamW 收敛速度更快且更加稳定, 适合 Transformer 架构的训练。

学习率 (Learning Rate): 设为 $5e-5$ 。配合预训练权重进行微调。

学习率调度器 (Scheduler): 采用 StepLR 策略, 每经过 7 个 epoch, 学习率衰减为原来的 0.1。

批量大小 (Batch Size): 320。

训练轮次 (Epochs): 14。

输入图片尺寸: 经过预处理裁剪后统一为 $224 * 224$ 。

3、实验结果

(1) 数值结果与客观评价

指标	结果数值	说明
Test Loss	0.1160	测试集上的损失值
Test Accuracy	0.9700	测试集上的分类精度

模型在独立的测试集上进行了最终评估, 结果如下表所示:

由结果可见, Swin Transformer 在该数据集上取得了极高的分类精度 (97.0%), 证明了该方法在遥感场景分类任务中的有效性。

(2) 可视化结果

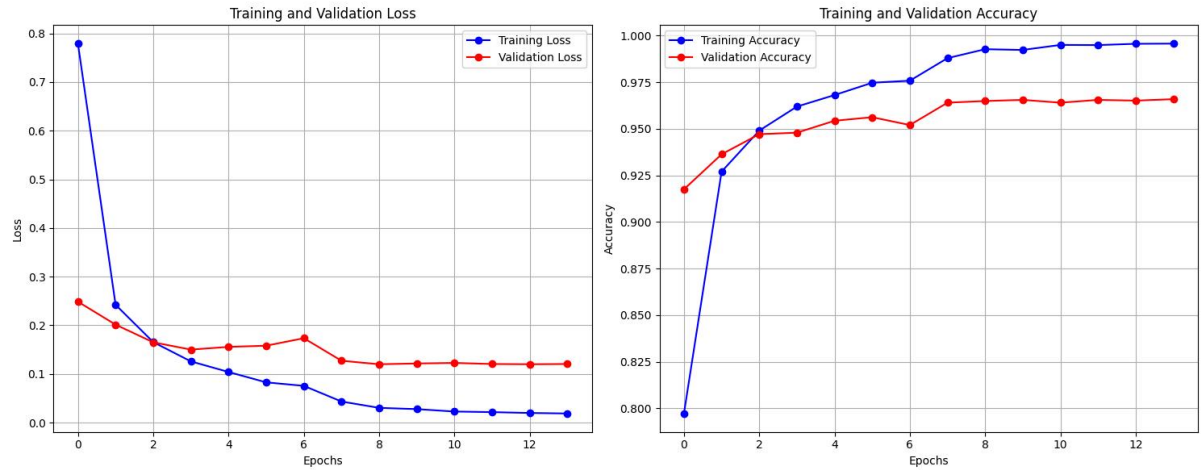


图 20 训练过程 Loss 与 Accuracy 变化曲线

训练集和验证集两个损失函数均快速下降并趋于平稳，且训练集和验证集两个准确性稳步上升，最终验证集准确性稳定在 0.96-0.97 左右。这表明模型收敛良好，未出现明显的过拟合现象。

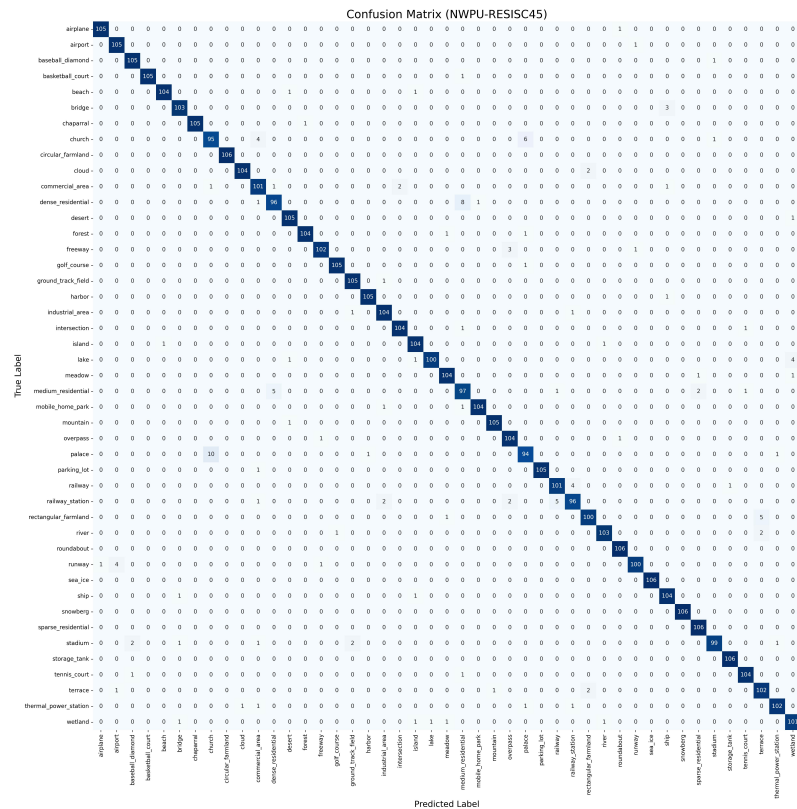


图 21 分类混淆矩阵

矩阵显示出极少的误分类情况，例如 church 或 palace 等类别之间可能存在少量的特征混淆，但整体分类效果非常清晰。